FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Tue 6 September 2016 - Part Theory

NAME (capital letters pls.):

MATRICOLA: SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
  1. Theory (80%): Syntax and Semantics of Languages
     - regular expressions and finite automata
     - free grammars and pushdown automata
     - syntax analysis and parsing methodologies
     - language translation and semantic analysis
  2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

1. Consider the regular expression R below, over the three-letter alphabet $\Sigma = \{\ a,\ b,\ c\ \}$.

$$R = (\ a\ \mid\ b\ )^*\ b\ (\ b\ \mid\ c\ )^*$$

Answer the following questions:

(a) Write all the strings x with $|\,x\,| \leq 2$ that belong to language $L\,(R)$, then say if the regular expression R is ambiguous and justify your answer.

(b) By using the Berry-Sethi method (BS), find a deterministic finite-state automaton A equivalent to the regular expression R above.

(c) By using a systematic method, find the finite-state automaton $\overline{A}$ complement of the automaton A previously found, and if necessary minimize the complement automaton $\overline{A}$.

(d) By using a systematic method, find a regular expression $\overline{R}$ equivalent to the complement automaton $\overline{A}$ previously found.

(e) (optional) Verify that the strings x of point (a) *do not belong* to the complement language $L\,(\overline{R})$ and shortly explain why.

---

2 Free Grammars and Pushdown Automata 20%

1. Consider a three-letter alphabet $\Sigma = \{\ a,\ b,\ c\ \}$, and the three languages L_1, L_2 and L as defined below. Answer the following questions:

   (a) Write a grammar G_1, *BNF* and unambiguous, that generates the language:

   $$L_1 = \left\{\ a^i\ b^j\ c^k \mid \quad i,\ j,\ k \geq 0\ \wedge\ k \leq i\ \right\}$$

   Draw the syntax tree for string $a^3\, b\, c^2 \in L_1$.

   (b) Write a grammar G_2, *BNF* and unambiguous, that generates the language:

   $$L_2 = \left\{\ a^i\ b^j\ c^k \mid \quad i,\ j,\ k \geq 0\ \wedge\ k \leq j\ \right\}$$

   Draw the syntax tree for string $a\, b^3\, c^2 \in L_2$.

   (c) Write a *BNF* grammar G that generates the language:

   $$L = \left\{\ a^i\ b^j\ c^k \mid \quad i,\ j,\ k \geq 0\ \wedge\ (\ k \leq i\ \vee\ k \leq j\ )\ \right\}$$

   Determine if grammar G is ambiguous and justify your answer: if it is ambiguous then provide a string with multiple syntax trees, otherwise argue that it does not admit any ambiguous string.

---

2. Consider a program written in a simplified C language, which features the following syntactic structures:

   - a program consists of a declaration block followed by an execution block, both mandatory
   - the declaration block defines integer variables declared as usual in the C language
   - the execution block is a list of the following statement types:
     - assignment with a variable on the left side and an arithmetic expression on the right side
     - for loop with an initialization clause (an assignment), an iteration clause (a relational expression) and an update clause (an assignment), which are grouped in round parentheses ‘ (’ ‘) ’ and separated by semicolons ‘ ; ’ as usual in the C language, followed by an execution block
     - a nested execution block

     and each statement is followed by a semicolon ‘ ; ’
   - an arithmetic expression may contain addition operators ‘ + ’, variables, integer constants and subexpressions delimited by round parentheses ‘ (’ ‘) ’
   - a relational expression consists of a comparison operator (‘ < ’, ‘ > ’, ‘ == ’, etc) and two arithmetic expressions to be compared
   - the for loops may be nested
   - the blocks are delimited by graph parentheses ‘ { ’ ‘ } ’ and may be empty; furthermore, the execution block may contain nested blocks
   - variable names and numerical constants can be schematized by the terminals id and num, respectively, to be left unexpanded

   For any detail that may have been left unspecified one can adopt the usual conventions of the C language. Here is a sample program:

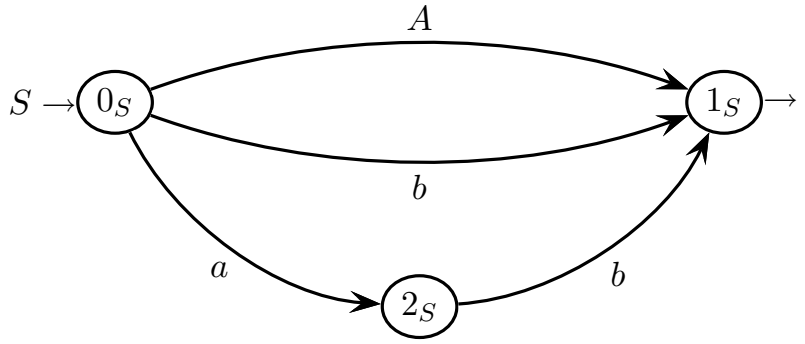
```
{ int a, b, c, d; }                        // declaration block

{                                          // execution block
  a = b + c + (a + d) + 1;                 // assignment statement
  for (a = 1; a + b < d; a = a + 1) {      // for loop
    b = b + 2;
    {                                      // nested execution block
      ...
    };
    c = d;                                 // assignment statement
  };
}
```

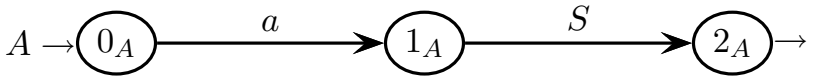
   Write a grammar G, *EBNF* and unambiguous, that models the sketched language.

---

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G, represented as a machine net over the two-letter terminal alphabet $\Sigma = \{\ a,\ b\ \}$ and the two-letter nonterminal alphabet $V = \{\ S,\ A\ \}$ (axiom S).





Answer the following questions:

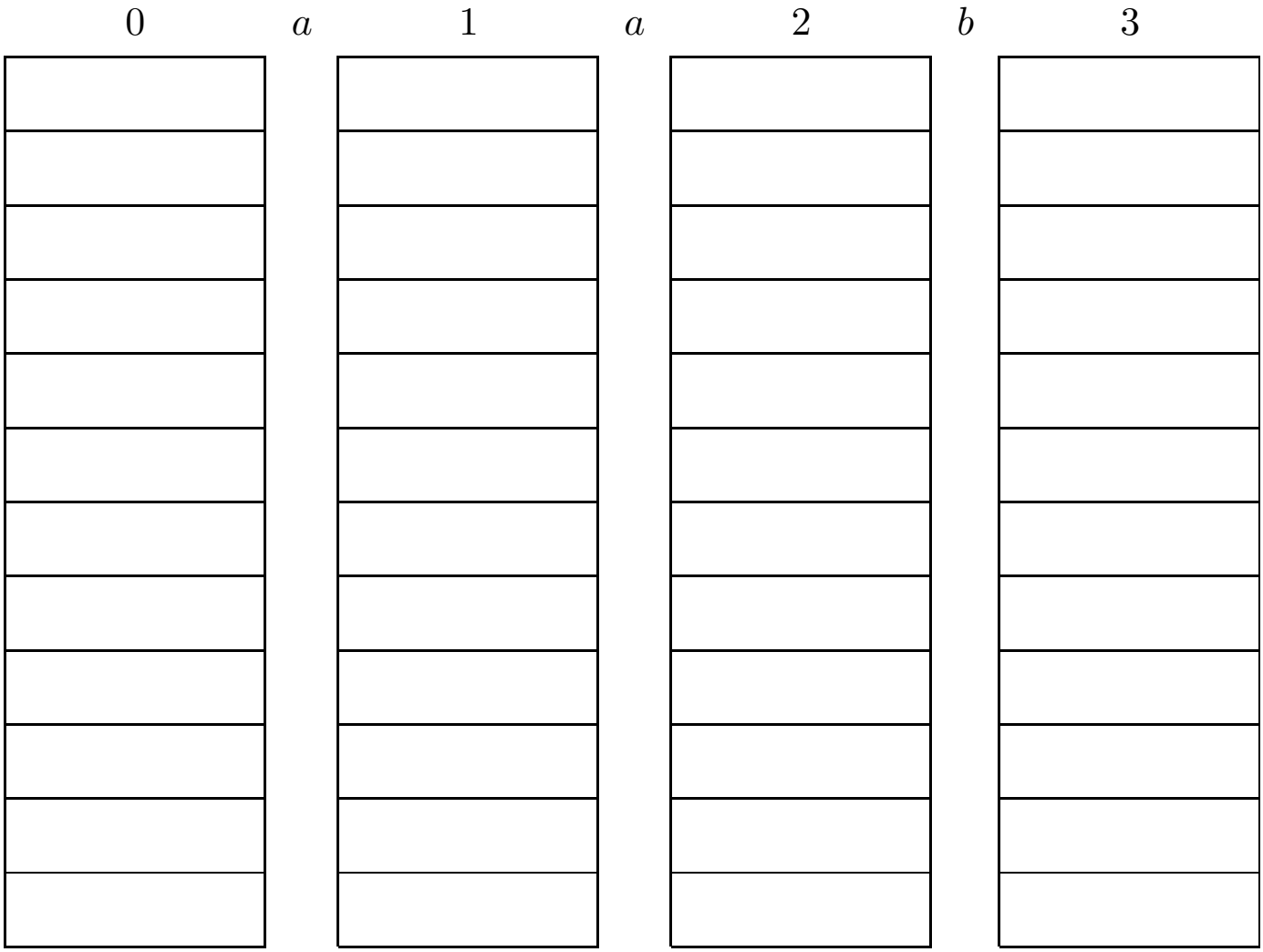
(a) Draw the syntax tree (or trees if there are two or more) of the valid string $a\,a\,b$.

(b) Draw the complete pilot of grammar G, say if grammar G is of type $ELR(1)$ and shortly justify your answer. If the grammar is not $ELR(1)$ then highlight all the conflicts in the pilot.

(c) Write all the guide sets on the arcs of the machine net (shift and call arcs, and final arrows), say if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer. If you wish, you can use the figure above to add the call arcs and annotate the guide sets.

(d) (optional) Analyze the valid string $a\,a\,b$ using the Earley method; show the item/all the items that gives/give evidence of string acceptance. Use the Earley vector prepared on the next page.

Earley vector of string $a\ a\ b$ (to be filled in)
(the number of rows is not significant)



4 Language Translation and Semantic Analysis 20%

1. Consider the source language L_s below, over the three-letter alphabet $\{\ a,\ b,\ c\ \}$:

$$L_s = \left\{\ a^m\ b^h\ c^k \mid \quad h + k = m\ \wedge\ h,\ k \geq 0\ \wedge\ m \geq 1\ \right\}$$

The following source grammar G_s, *BNF* and unambiguous, generates the source language L_s above:

$$G_s \begin{cases} S_1 \rightarrow a\ S_1\ c \\ S_1 \rightarrow a\ c \\ S_1 \rightarrow a\ S_2\ b \\ S_2 \rightarrow a\ S_2\ b \\ S_2 \rightarrow \varepsilon \end{cases}$$

Now consider a translation $\tau\colon L_s \rightarrow \{\ a,\ c\ \}^*$ that works as follows:

$$\tau\left(a^m\ b^h\ c^k\right) = a^h\ c^h$$

Answer the following questions:

(a) Write all the source strings $x \in L_s$ with $|\,x\,| \leq 4$, and for each of them write the corresponding translation $\tau\,(x)$.

(b) Find a suitable syntactic translation scheme (or grammar) G_τ, of type *BNF* and unambiguous, based on the given source grammar G_s, that computes the translation τ.

(c) (optional) Verify that the scheme (or grammar) G_τ is correct, by means of the sample (source, translation) string pairs $\big(x,\ \tau\,(x)\big)$ found at point (a). You can write the derivation or draw the syntax trees of each pair to be verified.

---

2. An e-commerce application computes the overall discount and the final total price for an ordered list of purchased items. Starting from the list head (on the left), the discount rate is 3 % for all the purchased items the total value of which is less than or equal to 100 euros, and it grows to 5 % for the following items in the list, if any.

   Thus, for instance, if the list contains items with prices equal to 20, 50, 40 and 30 euros, then the discount is equal to:

   $$20 \times 0.03 + 50 \times 0.03 + 40 \times 0.05 + 30 \times 0.05 = 0.6 + 1.5 + 2.0 + 1.5 = 5.6 \text{ EUR}$$

   and the final total price will be $140 - 5.6 = 134.4$ EUR.

   Notice that the order of the items in the list is relevant. For a list that contains items with prices equal to 20, 50, 30 and 40 euros, the discount is equal to:

   $$20 \times 0.03 + 50 \times 0.03 + 30 \times 0.03 + 40 \times 0.05 = 0.6 + 1.5 + 0.9 + 2.0 = 5.0 \text{ EUR}$$

   and the final total price will be $140 - 5 = 135$ EUR.

   The list of purchased items is modeled by the following abstract syntax (axiom S):

   $$\begin{cases} 1\colon\ S \ \rightarrow\ L \\ 2\colon\ L \ \rightarrow\ i\ L \\ 3\colon\ L \ \rightarrow\ i \end{cases}$$

   where the terminal symbol i represents a purchased item, having a real-valued attribute v (the functional attribute v is pre-computed during lexical analysis and is conventionally assumed to be right) that represents the item price in euros.

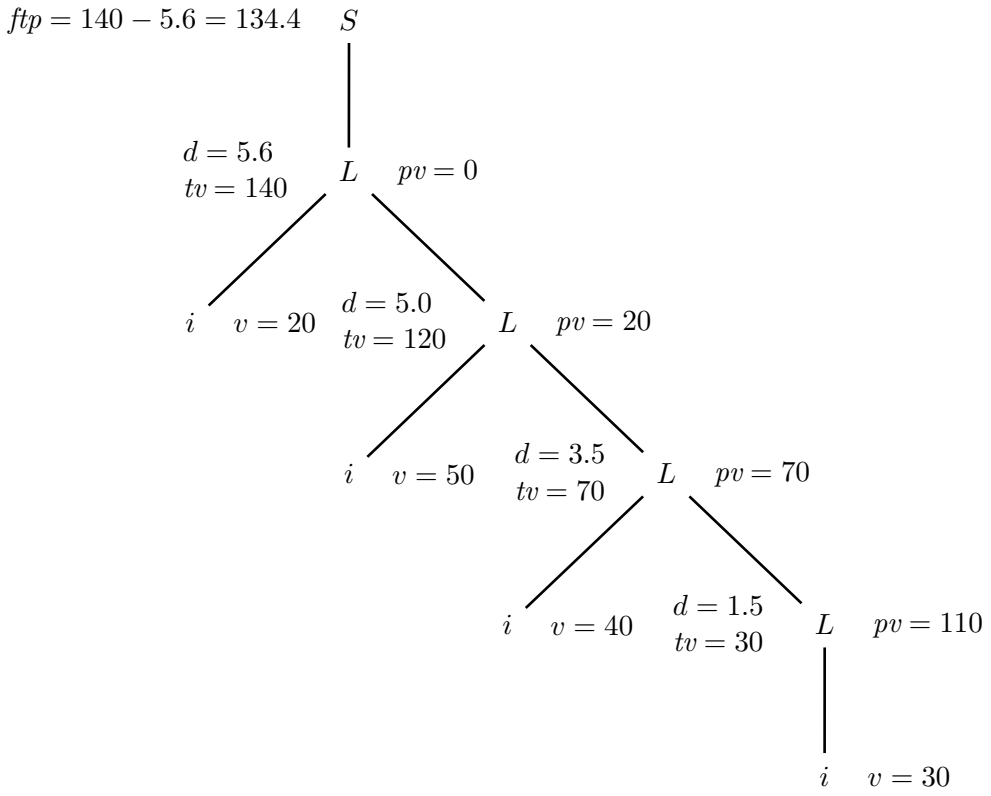
   The attributes to use are already assigned. See the table below. Other attributes are unnecessary and should not be added.

attributes assigned to be used

type	*name*	*domain*	*(non)term.*	*meaning*
right	pv	real	L	*previous value*: total value of the items occurring in the previous list positions
left	d	real	L	*discount*: discount for the items in the subtree rooted in L
left	tv	real	L	*total value*: total value of the items in the subtree rooted in L
left	*ftp*	real	S	*final total price* = total value of the items in the whole tree − overall discount
right	v	real	i	*value*: value of the item (this attribute of a terminal is pre-computed by lexical analysis)

Answer the following questions:

(a) Write an attribute grammar, based on the above syntax, that defines the computation of the final price *ftp* in the tree root, as explained above. It is suggested to use the attributes v, pv, d and tv with the meaning explained (see also the table above). An example for the item list [20, 50, 40, 30] is provided below.



(b) Decorate the syntax tree of the item list [20, 50, 30, 40] with the values of the attributes. Use the syntax tree prepared on the next page.

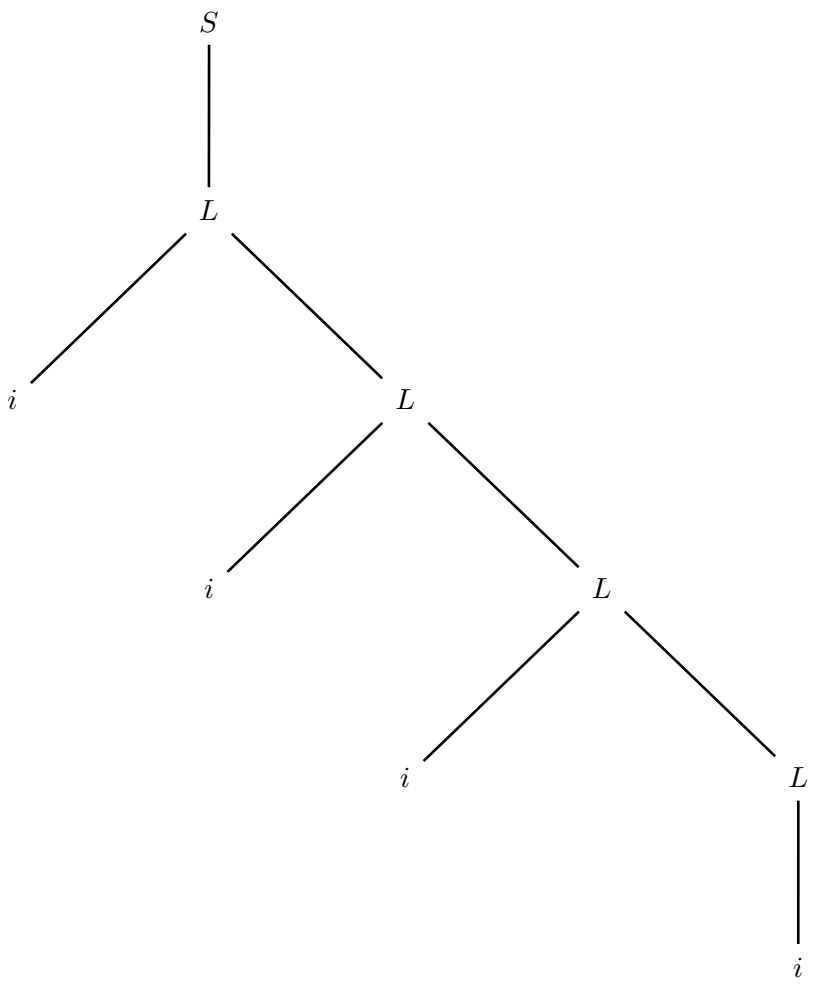
(c) Determine if the attribute grammar is of type one-sweep and if it satisfies the L condition, and reasonably justify your answers.

(d) (optional) Write the semantic evaluation procedure for the nonterminal L.

attribute grammar to write - question (a)
(for clarity also the terminals are indexed)

#	*syntax*	*semantics*
1:	$S_0 \rightarrow L_1$	
2:	$L_0 \rightarrow i_1\ L_2$	
3:	$L_0 \rightarrow i_1$	

syntax tree to decorate - question (b)



sone-sweep and L condition - question (c)

semantic evaluation procedure of nonterminal L - question (d)